

DESC

HISTORY

RULES

README

SETTINGS

Codewriting

<<

Problem Description

Given the pseudocode below to find a target number i between l and r (both inclusive):

```
while (l < r) {
    int mid = (l + r) / 2;
    if (i < mid) r = mid - 1;
    else if (i > mid) l = mid + 1;
    else {
        print("Found");
        break;
    }
}
```

You are required to answer q queries, where in each query, you will be provided with values l and r . For each query, determine the following:
What is the probability that the above code will fail to print "Found" when any value i (where $l \leq i \leq r$) is chosen? Express this probability as an integer where the fraction is $\frac{p}{q}$ and $\gcd(p, q) = 1$. You should compute $p * q^{-1} \bmod 10^9+7$, where q^{-1} denotes the multiplicative inverse of q modulo 10^9+7 .

Problem Constraints

$1 \leq A[i][0] \leq A[i][1] \leq 10^6$

Input Format

The first argument is a 2D integer array A , denoting the queries l and r in each row.

Output Format

Return an integer array denoting the probability in the form $p * q^{-1} \bmod 10^9+7$ for each query.

Example Input

Input 1:

$A = [[2, 9]]$

Input 2:

$A = [[10, 10], [10, 12]]$

DESC

HISTORY

RULES

README

SETTINGS

Input 1:
A = [[2, 9]]

Input 2:
A = [[10, 10], [10, 12]]

Example Output

Output 1:
[5000000004]

Output 2:
[1, 666666672]

Example Explanation

Explanation 1: The numbers 2, 4, 6, 9 will result in the code failing to print "Found". Thus, the probability is $4 / 8 = 1 / 2$, hence `1 * modInverse(2) = 500000004`.

Explanation 2:
Since `1` and `x` are equal in the first query, the loop won't run, so the probability is 1. Only 11 will be found in the second query, so the probability is `666666672`.

- [execution time limit] 0.5 seconds (cpp)
- [memory limit] 1 GB
- [input] array.array.integer a
- [output] array.integer

[C++] Syntax Tips

```
// Prints help message to the console
// Returns a string
string helloWorld(string name) {
    cout << "This prints to the console when you Run Tests" << endl;
    return "Hello, " + name;
}
```

C++

1

><

RU

DESC

HISTORY

RULES

README

SETTINGS

🔔 Codewriting

⏪

Problem Description

You are a book collector aiming to sell your collection of A books. The books are arranged in a line with the i th book to the left of the $(i + 1)$ th book for $1 \leq i \leq A - 1$. The thickness of each book is represented by an array B ($B[i]$ representing the thickness of the i th book) and each book has a unique thickness.

To enhance the appeal of your books, you can apply a special protective cover to some of them. However, this cover is expensive, so you want to minimize its use while ensuring the following conditions are met:

1. You should apply the protective cover to at least one book.
2. If you apply the cover to the i th book, then you must also apply it to each book that is thicker than the i th book.
3. There must exist at least one subarray (contiguous segment) of books of size at least C , such that the number of books with the protective cover is greater than the number of books without the cover.

Your task is to determine the smallest number of books on which you should apply the protective cover to satisfy the above conditions.

Problem Constraints:

- $1 \leq A \leq 10^5$
- $1 \leq B[i] \leq A$, ($1 \leq i < j \leq n$, $B[i] \neq B[j]$)
- $1 \leq C \leq A$

Input Format:

- First argument A is the number of books.
- Second argument B is an array representing the thickness of the books.
- Third argument C is the minimum length of the subarray according to the condition defined in the problem statement.

Output Format:

- Return a single integer representing the minimum number of books on which you will apply the protective cover.

Example Input:

Input 1:



DESC



HISTORY



RULES



README



SETTINGS

Example Input:

Input 1:

- A = 5
- B = [2, 3, 5, 1, 4]
- C = 3

Input 2:

- A = 4
- B = [2, 3, 1, 4]
- C = 2

Example Output:

Output 1:

- 2

Output 2:

- 2

Example Explanation:

For Input 1:

- We can apply the protective cover on books at index 3 and at index 5 (1-based indexing).
- Now, the subarray [3, 5] is of size at least 3, and it has a greater number of books with the protective cover compared to books without the cover.
- We can prove that this is the smallest number of books on which we can apply the protective cover.

For Input 2:

- We can apply the protective cover on books at index 2 and 4 (1-based indexing).
- Now, the subarray [2, 4] is of size at least 2, and it has a greater number of books with the protective cover compared to books without the cover.
- We can prove that this is the smallest number of books on which we can apply the protective cover.

<<

C++

1

><

⋮ MORE

VIEW DIFF

SUBMIT

NEXT

DESC

HISTORY

RULES

README

SETTINGS

• 2

<<

Example Explanation:

For Input 1:

- We can apply the protective cover on books at index 3 and at index 5 (1-based indexing).
- Now, the subarray [3, 5] is of size at least 3, and it has a greater number of books with the protective cover compared to books without the cover.
- We can prove that this is the smallest number of books on which we can apply the protective cover.

For Input 2:

- We can apply the protective cover on books at index 2 and 4 (1-based indexing).
- Now, the subarray [2, 4] is of size at least 2, and it has a greater number of books with the protective cover compared to books without the cover.
- We can prove that this is the smallest number of books on which we can apply the protective cover.

- [execution time limit] 0.5 seconds (cpp)
- [memory limit] 1 GB
- [input] integer a
- [input] array.integer b
- [input] integer c
- [output] integer

[C++] Syntax Tips

```
// Prints help message to the console
// Returns a string
string helloWorld(string name) {
    cout << "This prints to the console when you Run Tests" << endl;
    return "Hello, " + name;
}
```

C++

1

><

RU

DESC

HISTORY

RULES

README

SETTINGS

🔔 Codewriting

⏪

Problem Description

An array is considered valid if there is no subarray longer than B where all elements are identical. Additionally, the array can only contain integers from the range [1, C]. Determine the number of valid arrays of length A. Since the result can be very large, return it modulo (10^9+7).

Problem Constraints

- (1 <= A <= 10^9)
- (1 <= B <= min(50, A))
- (1 <= C <= 10^5)

Input Format

- The first argument is an integer A.
- The second argument is an integer B.
- The third argument is an integer C.

Output Format

- Return an integer representing the number of valid arrays.

Example Input

Input 1:

- A = 3
- B = 1
- C = 3

Input 2:

- A = 3
- B = 3
- C = 2

C++

1

2

><

RUN

DESC

HISTORY

RULES

README

SETTINGS

Example Output

Output 1:

- 12

Output 2:

- 8

Example Explanation

Explanation 1:

- The valid arrays are: [1, 2, 1], [1, 2, 3], [1, 3, 1], [1, 3, 2], [2, 1, 3], [2, 1, 2], [2, 3, 1], [2, 3, 2], [3, 1, 2], [3, 1, 3], [3, 2, 1], [3, 2, 3].

Explanation 2:

- All possible arrays of length 3 are valid arrays.
- [execution time limit] 2 seconds (cpp)
- [memory limit] 1 GB
- [input] integer64 a
- [input] integer b
- [input] integer c
- [output] integer64

[C++] Syntax Tips

```
// Prints help message to the console
// Returns a string
string helloWorld(string name) {
    cout << "This prints to the console when you Run Tests" << endl;
    return "Hello, " + name;
}
```

⋮ MORE

VIEW DIFF SUBMIT NEXT

C++

1

2

><

▶ RU

DESC

HISTORY

RULES

README

SETTINGS

 Codewriting



Problem Description

You are given an array `arr` consisting of `N` integers. You are also given `q` queries. Each query consists of three integers `x`, `y`, and `z`.

For each query:

1. Find the largest contiguous subarray `B` starting from index `x`, whose `y`th bit is set.
2. Update each of its elements `Bj` with `Bj ^ z`.

Where `^` denotes the bitwise XOR operator.

Your task is to print the total number of updates performed after `q` queries.

Given a 2D array `mat` that represents the queries:

- `mat[i][0] = x`
- `mat[i][1] = y`
- `mat[i][2] = z`

Problem Constraints

- $1 \leq N \leq 10^5$
- $0 \leq arr[i] \leq 10^9$
- $1 \leq X \leq N$
- $1 \leq Y \leq 30$
- $0 \leq Z \leq 10^9$
- $1 \leq q \leq 10^5$

Input Format

- First argument is an integer array `arr`.

C++

1

 RUN

Input Format

<<

- First argument is an integer array `arr`.
- Second argument contains 2D integer array `mat`.

Output Format

Return the number of updates performed after `q` queries.

Examples

Input 1:

```
arr = [2, 4, 3, 5, 4]
mat = [[3, 1, 4], [2, 3, 7]]
```

Output 1:

```
4
```

Input 2:

```
arr = [1, 4]
mat = [[2, 1, 3]]
```

Output 2:

```
0
```

Output 2:


```
0
```

Explanation

Explanation 1:

1st Query (3, 1, 4):

- {3,5} is the largest subarray having the 1st-bit set and starting with index 3.
- Array becomes $[2, 4, 3 \wedge 4, 5 \wedge 4, 4] = [2, 4, 7, 1, 4]$.

2nd Query (2, 3, 7):

- {4} is the largest subarray having the 3rd-bit set and starting with index 2.
- Array becomes $[2, 4 \wedge 7, 7 \wedge 4, 1, 4] = [2, 3, 3, 1, 4]$.

Explanation 2:

1st Query (2, 1, 3):

The length of the maximum contiguous subarray that satisfies the given condition is 0, so there is no change in the array.

- [execution time limit] 1 seconds (cpp)
- [memory limit] 1 GB
- [input] array.integer arr
- [input] array.array.integer mat
- [output] integer

[C++] Syntax Tips

C++

1

 RUN